NATIONAL UNIVERSITY OF LESOTHO
DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

CS4433 – SOFTWARE ENGINEERING

MODERATED LEARNING MANAGEMENT SYSTEM

FINAL SOFTWARE ENGINEERING PROJECT REPORT

K. Mothosele
Student Number: 202322795

Department of Mathematics and Computer Science
National University of Lesotho

November 2026

Github Repository: https://github.com/ralvetech/Learning-Management-System

Abstract
This report presents the planning, requirements engineering, system design, implementation approach and remaining lifecycle work for a Moderated Learning Management System (LMS) developed as the CS4433 Software Engineering individual project at the National University of Lesotho. The proposed system addresses a secondary school's need to manage users, departments, courses, enrolments, assignments, submissions, assessment, moderation and the controlled release of marks. It is designed as an offline-capable C++17 desktop application using Qt 6 for the graphical interface, SQLite for local persistence and CMake as the build system.

The central business objective is to make assessment administration more consistent and auditable while preserving the school's requirement that students must not see marks before the Head of Department (HoD) has reviewed and released them. Four roles are supported: Administrator, Lecturer, HoD and Student. The system separates lecturer marking from HoD moderation, applies a defined late-submission penalty, supports extensions and resubmissions, and is intended to retain an audit history of mark changes.

The report consolidates the eight project phases into one narrative. It summarises the feasibility and project plan, stakeholder analysis and requirements, layered architecture and database design, implementation structure, verification strategy, configuration management, deployment planning and the CR1 maintenance request. It distinguishes completed design and implementation work from activities for which evidence is still required. In particular, no unverified test coverage, continuous-integration result, static-analysis metric, formal acceptance result or completed v1.1 release is claimed.

Table of Contents
1. 1. Introduction and Project Context
2. 2. Project Planning, Feasibility and Management
3. 3. Requirements Engineering
4. 4. System Design
5. 5. Implementation
6. 6. Verification and Validation
7. 7. Configuration Management and Quality Assurance
8. 8. Deployment and User Operation
9. 9. Maintenance and Change Request CR1
10. 10. Professional, Legal and Ethical Considerations
11. 11. Conclusion and Recommendations
12. References

1. Introduction and Project Context
Schools manage large volumes of assessment information. A typical assessment process involves creating a course assignment, informing enrolled students, receiving submissions, recording marks and feedback, checking results and communicating them to students. When these activities are handled through separate files or informal processes, it becomes difficult to maintain consistent records, apply deadlines fairly, identify the latest submission, control access to marks and determine who changed an

The Moderated LMS is intended to provide a single application for the core assessment workflow. It is not designed to replace every school information system. The project scope is deliberately limited to user and course administration, assignment management, submissions, marking, moderation and mark release. The assignment excludes online payments, video conferencing, automated grading, plagiarism detection and web or mobile interfaces.

The main organisational requirement is that a student must not see a mark immediately after a lecturer enters it. The Principal's position is that the HoD must check the result first. The lecturer's desire for prompt feedback is accommodated by allowing marks and written feedback to be entered and saved as provisional records. The HoD then reviews the departmental moderation queue, approves or returns a mark, and releases an approved result.

The second major business rule concerns late submissions. The HoD's statement specifies a deduction of ten per cent of the available mark for each late day, for up to three days, after which the submission receives zero. The student representative also states that late work should not simply be rejected. The selected interpretation is therefore to accept late work within the permitted period and calculate a penalty, while applying zero after the three-day window. The requirements documents identify an ambiguity about whether a partial day counts as a full day.

The system is intended to operate on a school computer or local network where internet access may be unreliable. This led to the selection of a desktop application with a local SQLite database. The application should not require a cloud service for core functions. The intended users include staff who may not be technical specialists, so the interface should present clear role-specific tasks, simple navigation and meaningful validation messages.

2. Project Planning, Feasibility and Management
The project follows the Waterfall development model specified in the assignment, with verification and validation organised according to the V-Model. The seven-week schedule divides the work into planning, requirements engineering, system design, implementation, verification and validation, deployment, maintenance and final completion. Each phase produces a defined artefact and provides a basis for the next phase.

The planning phase assessed technical, operational, economic, legal and schedule feasibility. Technical feasibility was considered positive because C++17, Qt, SQLite and CMake are established technologies suitable for a local desktop application. SQLite is lightweight and serverless, making it appropriate for a single-computer installation and offline use. CMake provides a repeatable build configuration, while UML tools, GoogleTest or Catch2, and static-analysis tools such as cppcheck or clang-tidy are available for the engineering activities.

Operational feasibility was assessed by considering the work of each stakeholder. Administrators need to maintain accounts, departments, courses, lecturer assignments and student enrolments. Lecturers need to create assignments, set deadlines and maximum marks, grant individual extensions, review submissions, mark work and provide written feedback. HoDs need oversight of marks in their departments and control over approval and release. Students need to see their enrolled courses and assignments, submit or resubmit work and view released marks and feedback. The system aligns with these responsibilities and replaces a fragmented process with a role-specific workflow.

Economic feasibility was rated positively because the development tools are free or open source and the plan assumes the school can use its existing computers. The planning estimate records M0 for software licences and hardware, approximately seven weeks of student development time, minimal training and low maintenance costs. No external labour or cloud infrastructure costs are included. These figures describe the academic project's assumed cost model, not a full institutional procurement estimate.

Legal feasibility was assessed in principle through data protection, assessment-record security, accessibility and fairness. The system stores learner identities, enrolments, submissions, marks and feedback, so access controls and careful retention are necessary. The plan proposes encrypted or otherwise protected storage, role-based access, audit logging and data minimisation. The precise legislation applicable to the school's deployment must be confirmed with the institution; this report does not claim legal certification.

Schedule feasibility was considered positive but dependent on strict time management. The original schedule allocates approximately one week to planning, one to requirements, one to design, one to implementation, one to verification and validation, one to deployment and one to maintenance and completion. The project plan identifies milestones for the SRS and use cases, design approval, implementation completion, V&V, v1.0, v1.1 and final submission on 7 November 2026.

The work breakdown groups the work into requirements and stakeholder analysis, UML and database design, implementation of authentication and role dashboards, assessment workflows, automated testing, release preparation and maintenance. The planning artefact includes a Gantt chart and milestone structure. The critical dependency is that the design and database structure must be stable before implementation.

Risk management was based on likelihood and impact scores from one to five, with exposure calculated as likelihood multiplied by impact. The highest recorded risk is design-phase overrun, with likelihood four, impact five and exposure twenty. Its mitigation is timeboxing, regular design review and prioritising essential diagrams; the contingency is to reduce design scope to the diagrams required by the assignment. Scope creep has likelihood three and impact four, exposure twelve, and is mitigated by freezing requirements after Gate 1 and using change control.

The overall planning recommendation was to proceed because the project was considered feasible across the five dimensions. The recommendation is conditional on maintaining the agreed scope, completing the phase-gate reviews, testing security-sensitive behaviour and being transparent about any work that remains incomplete. The planning phase therefore provides both the justification for the project and the controls for managing it.

3. Requirements Engineering
Requirements engineering began by identifying the stakeholders and analysing their interests, concerns and possible conflicts. The principal or school leadership prioritises simplicity and the protection of assessment results before HoD review. Lecturers need to create assignments, manage deadlines,

grant individual extensions, mark work and give feedback. HoDs need to review all departmental marks and apply the late policy. Students need access to enrolled-course information, submission and resubmission opportunities and written feedback.

The most significant conflict concerned mark visibility. The lecturer wanted students to receive marks immediately after marking, while the Principal required HoD checking before release. The resolution is to distinguish marking from publication. A lecturer can save a provisional mark and feedback, but this does not make the result visible to the student. The HoD performs moderation and explicitly releases the approved result.

A second conflict concerned late work. Students should be allowed to submit late work rather than having it automatically rejected, but the HoD specified a penalty and a maximum period. The proposed rule accepts work up to three late days, applies a deduction of ten per cent of the assignment's maximum mark for each late day, and gives zero beyond the permitted window. The SRS also records the ambiguity about whether partial days are rounded up or whether calendar days are used.

The requirements also clarify the relationship between extensions and deadlines. An authorised individual extension changes the effective deadline for the selected student. Late calculation must use that effective deadline rather than the original assignment deadline. The original requirements permit resubmission before the deadline but do not fully specify version handling. The selected design retains submission versions and identifies the latest valid version received before the effective deadline. This preserves an audit trail and avoids silently overwriting earlier work.

The Software Requirements Specification uses uniquely identified functional requirements and non-functional requirements, prioritised using MoSCoW and linked to verification methods. The functional requirement areas include authentication, role-based permissions, account administration, department and course maintenance, lecturer assignment, student enrolment, assignment creation, individual and class extensions, assignment listing, submission, resubmission, late-penalty calculation, marking and feedback, moderation, release and audit history. The requirements are intended to be verifiable through positive, negative and boundary tests.

The non-functional requirements address security, performance, usability, reliability, maintainability, portability and offline operation. The IT officer's performance concern should be converted into measurable acceptance criteria, such as response-time targets for login, course and assignment queries with a test dataset representing 800 stored student accounts. Security requirements include salted password hashes, protected authentication, role-based access and parameterised database queries. Reliability requirements include safe handling of invalid input and database errors, persistent storage and recovery from a controlled backup. Usability requirements include clear labels, simple role-specific navigation and useful error messages.

Use cases provide the bridge between stakeholder goals and the implementation. The four primary actors are Administrator, Lecturer, HoD and Student. Administrator use cases include managing accounts, departments and courses, assigning lecturers and enrolling students. Lecturer use cases include creating assignments, granting extensions, viewing submissions, marking and providing feedback. HoD use cases include reviewing departmental marks, approving or returning marks and releasing approved results. Student use cases include viewing enrolled courses and assignments, submitting and resubmitting work and viewing released marks.

Requirements traceability is necessary because the same business rule appears in several artefacts. The late-penalty requirement must map to the use case, class or strategy design, database values, implementation method and boundary tests. The mark-release requirement must map to the moderation workflow, mark status model, role permissions, database state and student visibility tests. The department requirement must map to user and course relationships, HoD filtering and integration tests.

4. System Design

The selected architecture is layered. The presentation layer provides role-aware screens, menus, prompts and feedback. The application or service layer coordinates use cases such as authentication, enrolment, submission, marking, moderation and release. The domain layer represents users, assignments, submissions, marks and business policies. The repository or Data Access Object (DAO) layer performs SQL operations, parameter binding, row mapping and transaction management.

Layering was selected because the application has distinct user interaction, business rules and persistence responsibilities. It allows business logic to be tested without requiring the full GUI or a live database, and it limits the impact of a database change on the rest of the application. Parameterised SQL can be centralised in repositories, while application services can coordinate transactions and enforce the correct sequence of operations.

The design-level class model uses an abstract User class to hold common identity and authentication responsibilities. Student, Lecturer, HeadOfDepartment and Administrator inherit from User and provide role-specific operations. This supports polymorphism and reduces duplicated identity logic. Assignment stores the course, title, deadline and maximum mark. Submission stores the assignment, student, submission timestamp, status and version. MarkRecord stores the score, feedback, status and release information.

The Strategy pattern is used for late-penalty calculation. A LatePenaltyStrategy interface defines the calculation contract, while StandardLatePenalty implements the school's current rule. The strategy isolates the policy from the submission workflow. If the school later changes the percentage, penalty limit or treatment of partial days, a different strategy can be introduced without rewriting unrelated submission and marking code. The Repository or DAO pattern separates persistence from domain logic.

The design aims to follow SOLID principles. Single Responsibility is supported by separating presentation, application services, domain rules and persistence. Open/Closed is supported by the penalty strategy interface, which allows alternative policy implementations without changing the calling workflow. Liskov Substitution is supported when role subclasses honour the common User contract. Interface Segregation encourages small repository and service interfaces rather than one large interface containing unrelated operations.

Behavioural models describe the lifecycle of a submission and mark. A submission may move through states such as Submitted, Late, Marked, Moderated and Released. The state machine prevents invalid transitions, for example a student should not see a mark that has not been released. The activity diagram with swimlanes describes the end-to-end process among Student, System, Lecturer and HoD. The sequence diagram for Submit Assignment includes validation, deadline evaluation, persistence and the late or resubmission alternatives.

The database design is intended to be normalised to at least Third Normal Form. The principal entities include users, departments, courses, lecturer-course assignments, student enrolments, assignments, submissions, individual extensions, marks and mark-audit records. A department has multiple courses, while each course belongs to one department. Students and courses have a many-to-many relationship resolved through an enrolment

table. Courses have multiple assignments; assignments have multiple submissions; each submission belongs to a student and an assignment. Marks and audit entries are linked to submissions and users.

The class model and database model are not identical because they serve different purposes. Inheritance in the object model represents shared behaviour among user roles, while the relational database can store common user fields in one users table with a role attribute and additional role-specific fields where required. Many-to-many relationships require linking tables even when the object model presents collections.

5. Implementation

Implementation translates the approved requirements and design into the working C++ application. The recorded technology stack is C++17, Qt 6, CMake and SQLite. Qt 6 provides the desktop graphical interface, C++17 implements application and domain behaviour, SQLite stores local records and CMake manages the build.

The application entry point in src/main.cpp starts the Qt application and configures the interface. Database-related operations are managed through src/database.cpp. Authentication and navigation are separated into login-dialog and main-window modules. The administrator, lecturer, HoD and student dashboards are implemented in separate UI modules. Shared definitions are placed in common and domain headers.

The Administrator dashboard supports management of users, departments, courses, lecturer assignments and student enrolments. The administrator can view records, filter the displayed dataset and search visible records. The enhanced administration interface also includes operations such as creating users, adding departments, creating courses, assigning lecturers, enrolling students, editing selected records, removing selected enrolments and exporting table data to CSV.

The Lecturer workflow covers assignment creation and the handling of student submissions. A lecturer should access only courses assigned to that lecturer, create assignments with a title, instructions, deadline and maximum mark, and grant permitted individual extensions. The lecturer reviews submissions, enters a raw mark and written feedback and submits the mark for moderation.

The HoD dashboard provides a moderation queue for submissions in the HoD's department. The queue is intended to show the submission identifier, course, assignment, student, submission date, status, raw mark, final mark and feedback. The HoD can moderate a selected mark, release a selected mark and view the mark audit. The project development evidence identified an empty HoD queue when department identifiers in user, course or submission records did not match. This demonstrates why referential integrity and department-scoped query testing are important.

The Student dashboard is intended to display the student's enrolled courses and their assignments, support submission and permitted resubmission, and show released marks and feedback. The student must not see a provisional mark. The interface should clearly distinguish assignment deadlines, submission status and mark release status. The class-wide extension request in Phase 8 also requires the revised effective deadline to appear in the assignment list for every affected student.

The late-penalty strategy represents a deduction of ten per cent of the assignment maximum per late day for up to three days, with zero after more than three days. For example, if the maximum mark is 100 and the raw mark is 80, the proposed rule gives a final mark of 80 on time, 70 after one late day, 60 after two late days and 50 after three late days. After the three-day allowance the final mark is zero.

A development issue was identified around the display and persistence of the final mark. The database query evidence showed records with raw_mark and final_mark values, including one record with a raw mark of 98 and a final mark of 78, which is consistent with a 20-point deduction on a 100-point maximum. However, the existence of a different final_mark value in SQLite does not by itself prove that the HoD or student interface displays the adjusted value correctly.

The application uses SQLite for local persistence. The database records include user accounts and department identifiers, and development checks have used SQL queries to inspect submissions, marks and course relationships. A department mismatch was identified during development and a correction was proposed to align the lecturer and HoD department_id values with the relevant course or submission department.

Security requirements include unique usernames, authentication before protected access, role-based permissions and salted password hashes. The database should never store plain-text passwords. The final report and implementation evidence should identify the password-hashing library and configuration actually used. The application should validate all user input, enforce mark bounds, check that lecturers are assigned to the course and ensure that HoDs moderate only their own department.

The available project evidence confirms a modular application structure and screenshots of the administrator and HoD interfaces, as well as database inspection through SQLite Browser. The Phase 4 documentation also records the technology stack and the intended role workflows. It explicitly identifies the need to attach clean build output, screenshots of all role dashboards, database schema and sample records, end-to-end workflow results, late-penalty boundary tests and evidence that unreleased marks are hidden.

6. Verification and Validation

Verification determines whether the implementation conforms to the SRS and design. Validation determines whether the system meets the actual needs of the school stakeholders. The assignment adopts the V-Model, which links each development artefact to a corresponding test level. Requirements are checked through acceptance scenarios; architecture and component interactions are checked through system and integration tests; and domain classes and algorithms are checked through unit tests.

Unit testing should focus on isolated business rules and domain classes. GoogleTest or Catch2 can test the late-penalty strategy, mark validation, status transitions, role permission decisions and effective-deadline calculation. Boundary value analysis is particularly important for deadlines and penalties. Test cases should include a submission before the deadline, exactly at the deadline, one second after the deadline, close to a 24-hour boundary, one day late, three days late and more than three days late.

Integration testing should examine the boundaries between the Qt application, application services, repositories and SQLite. Tests should use a separate temporary database rather than the real project database. They should verify that an account can be authenticated, a course can be retrieved, an assignment can be saved, a student submission can be persisted and reloaded, a mark can be updated and an audit record can be created.

System tests should exercise complete workflows derived from the use cases. One scenario begins with the administrator creating a course, assigning a lecturer and enrolling a student. The lecturer creates an assignment, the student submits work, the lecturer marks it and the HoD moderates and releases the result. The student then verifies that the released mark and feedback are visible.

Acceptance testing should involve representative users or a clearly identified role-play if stakeholders are unavailable. The administrator should be able to maintain the school records; the lecturer should be able to manage assignments and marking; the HoD should be able to review and release departmental marks; and the student should be able to submit work and view only permitted information. Feedback should be recorded and classified.

The automated test suite should be executed by the build system and, where configured, by a continuous-integration pipeline. The assignment sets a target of at least 70 per cent line coverage of the business-logic layer and requires the actual figure to be reported. Coverage should be measured with a tool such as gcov/lcov and should state the number of covered lines, total lines, tool version and command used.

A defect log should record the defect identifier, severity, description, cause, correction, retest result and status. Critical defects include data loss, a security breach or inability to use a core function. High defects affect a major workflow, such as a moderation queue that fails to show eligible marks. Medium defects have a workaround but produce incorrect behaviour, while low defects are minor presentation or usability issues.

7. Configuration Management and Quality Assurance

Git is the configuration-management tool for controlling changes to source code, headers, database schema, build configuration, tests and documentation. The project identifies a hosted repository at https://github.com/ralvetech/Learning-Management-System. A stable main branch should contain the latest reviewed state. Short-lived feature branches should be used for new functionality, while fix branches should isolate defect corrections.

A release tag identifies a reproducible version of the source. The project plan anticipates v1.0 for the initial release and v1.1 for the maintenance change. A release should be created from a clean, known commit and accompanied by release notes describing the version, features, database changes, known issues and verification evidence. Build directories, object files, generated binaries, local test databases, credentials and real learner records should not be committed. The development log shows that Git initially refused a commit because no author name or email had been configured.

Continuous integration is intended to configure, build and test the project on every push and pull request. The proposed GitHub Actions workflow checks out the repository, installs CMake, GCC, Ninja, Qt 6 and SQLite development dependencies, configures the project, builds it and runs CTest. This workflow is a configuration proposal in the supplied phase documentation. It must not be described as active or green unless a successful run is available in the repository.

Quality assurance includes static analysis, code metrics and self-review. cppcheck or clang-tidy should be run against the source, and findings should be reviewed for resource management, unchecked return values, unsafe SQL construction, input validation, duplicated logic, dead code and high-complexity functions. The report should include the tool version, command or configuration, findings summary and corrections. Lines of code and cyclomatic complexity should be measured for key functions such as authentication, submission, marking and moderation.

The current project material describes the QA activities and provides a sample CI configuration, but it does not contain actual static-analysis results, measured code metrics, completed code-review records or CI execution evidence. Those items remain to be produced from the final repository.

8. Deployment and User Operation

The deployment model is a school workstation containing the Moderated LMS desktop application and a local SQLite database. Within the application, the deployment diagram should show the Qt 6 presentation layer, C++ application and domain logic, and SQLite data-access layer. The database stores users, departments, courses, enrolments, assignments, submissions, marks, moderation and release state, and audit records.

The baseline deployment is a single authorised workstation with a local database. If several computers must access the same records concurrently, the school must review the architecture and database access model. SQLite is suitable for lightweight local persistence, but a shared database file on a network folder should not be assumed safe for concurrent multi-user operation without testing and an approved locking and backup design.

A school technician's installation procedure should begin by confirming the operating-system and hardware requirements. The technician installs the required Qt 6 runtime libraries and other dependencies identified by the release build, obtains the approved package, extracts the application into a controlled application directory and creates a separate data directory with restricted permissions. The SQLite database is initialised or restored into the data directory, and the application is checked to ensure that it uses the intended database path.

Backup and recovery are essential because the local SQLite database contains academic records. Backups should be performed on an agreed schedule and before upgrades. A safe SQLite backup method or controlled shutdown procedure should be used; copying a database during active writes without a safe method can create an unreliable backup. Access to database and backup folders should be restricted. Backups should be stored separately from the workstation and restoration should be tested periodically using a copy.

The role-specific user guidance follows the application's permissions. Administrators sign in to manage users, departments, courses, lecturer assignments and enrolments, and should verify department/course relationships before saving. Lecturers access assigned courses, create assignments, set deadlines and maximum marks, grant permitted extensions, review submissions and enter marks and feedback. HoDs review departmental queues, check raw and final marks, moderate with a reason and release approved marks.

The v1.0 release checklist requires Must requirements to be verified or recorded as limitations, critical and high defects to be resolved or deployment withheld, database initialisation and upgrade instructions to be included, user manuals to be prepared, backup and restore to be tested and the release to be built from a clean tagged revision. The installation guide, deployment diagram, role manuals and release notes are required deliverables.

9. Maintenance and Change Request CR1

After the initial release, the school submitted CR1: a lecturer should be able to extend the deadline for a whole class, not just an individual student, and the extension should appear on the student's assignment list. This is a functional enhancement and is classified as perfective maintenance because it improves an existing capability and extends the system to support a broader workflow.

The impact analysis identifies several affected artefacts. The SRS must include a class-wide extension requirement and a requirement to display the effective deadline to affected students. The use-case model must update the lecturer's extension use case and the student's assignment-list use case. The class design must update the assignment or extension model and the relevant service and repository operations. The database design must store the class-level extension or revised assignment deadline and record who made the change and when. The lecturer interface needs a control to set or update the class extension.

The business rules state that only an authorised lecturer assigned to the relevant course or assignment may set a class extension. The new deadline must be valid and should not be earlier than the original deadline unless an authorised correction is intended. The system should record the actor, timestamp and, where required, a reason. Every affected student should see the effective deadline. Submission and late-penalty logic must use the effective deadline rather than the original deadline.

Implementation should take place on a separate feature branch. The database schema should be updated through a migration or versioned initialisation script so that an existing v1.0 database can be upgraded without losing records. The lecturer workflow, student assignment query and shared effective-deadline calculation should be updated together. The migration should first be tested against a copy of a v1.0 database.

The maintenance documentation defines the intended v1.1 procedure and regression test set. However, the supplied evidence does not establish that CR1 has been fully implemented, migrated, tested or tagged as v1.1. The final report therefore records CR1 as the required maintenance change and explains its implementation and verification approach, rather than claiming a completed release without evidence.

10. Professional, Legal and Ethical Considerations

The LMS processes information that can affect learners' privacy, academic progress and reputation. User identities, enrolment information, submissions, marks and feedback must be treated as sensitive educational records. The school should collect only information needed for course administration and assessment, restrict access according to role and retain records only for an approved period.

Passwords must never be stored in plain text. The implementation should use a recognised password-hashing library with a unique salt and an appropriate work factor, and the report should identify the actual algorithm and configuration used. SQL statements should be parameterised to reduce injection risk. Role permissions must be checked in application logic for every protected operation, not merely represented by hidden buttons. Authentication, permission failures and mark changes should be handled consistently.

Assessment integrity requires a separation between marking and release. The lecturer's mark remains provisional until moderation, and only an approved mark may be released. Every mark change should record the actor, timestamp, previous value, new value and reason. The audit record supports accountability and dispute resolution. Related database updates should use transactions so that a mark and its audit entry cannot become inconsistent.

Fairness is especially important in the treatment of late work and extensions. The school should publish the penalty rule, define whether a partial day counts as a full day, explain the maximum late period and define how authorised extensions interact with class-wide extensions. The same policy should be applied consistently to comparable cases. The system should retain extension authorisation details and submission history so that decisions can be reviewed.

Accessibility and usability are professional responsibilities. The interface should use readable text, sufficient contrast, consistent navigation, clear labels and meaningful validation messages. Status must not be communicated by colour alone; labels such as Submitted, Late, Marked, Moderated and Released should be visible. Controls should be keyboard-operable where practical, and errors should explain what the user can do next.

Professional responsibility also requires honest reporting of software quality. The developer should not claim that tests passed, coverage targets were achieved, a release was deployed or security controls were verified unless evidence supports those statements. Known defects should be recorded and corrected according to severity. Changes to assessment rules should be approved by the school and reflected in the SRS, design, code and tests.

11. Conclusion and Recommendations

The Moderated LMS addresses a defined school need: to manage assessment workflows in a controlled, traceable and offline-capable application. The project planning phase found the solution feasible within the academic scope, using existing school hardware and free development tools. Requirements engineering resolved the conflict between immediate lecturer feedback and mandatory HoD moderation by separating provisional marking from explicit release. It also clarified the late-submission policy, individual extensions, resubmission history and role-based access.

The implementation uses C++17, Qt 6, CMake and SQLite and is organised into modular source files for application startup, database access, authentication, navigation and role-specific dashboards. The intended workflows cover administration, lecturer marking, HoD moderation and student submission and mark viewing. Development evidence identified important integration checks, particularly consistent department identifiers in moderation queries and the correct display of the penalty-adjusted final mark.

The remaining lifecycle work is defined through a V-Model test strategy, GoogleTest or Catch2 automation, system and acceptance scenarios, Git configuration management, CI, static analysis, deployment preparation and CR1 maintenance. The project documentation sets a 70 per cent business-logic line-coverage target and identifies the evidence required for v1.0 and v1.1.

In conclusion, the project establishes a coherent design and implementation direction for a moderated assessment system. Its strongest contribution is the explicit separation of lecturer marking, HoD moderation and student release, supported by auditability and a consistent late-penalty policy. The system should proceed to formal verification and stakeholder evaluation only after the identified integration issues are resolved.

Appendix A: Evidence status to be completed before submission

Evidence item	Current status in supplied material	Required final evidence
Phase 4 clean build	Implementation structure and screenshots are described; no complete build log supplied.	Clean configure/build output, compiler and dependency versions.
GoogleTest / Catch2	Test strategy and example test cases are documented; actual full run not supplied.	Test source, command output and passed/failed counts.
Business-logic coverage	Target of at least 70% identified; actual percentage not supplied.	gcov/lcov report with covered and total lines.
System and acceptance tests	Scenarios documented; signed stakeholder acceptance not supplied.	Completed test records, tester/date/build and defects.
CI pipeline	GitHub Actions example supplied; successful pipeline evidence not supplied.	Repository workflow file and successful run URL/screenshot.
Static analysis and metrics	Tools and metrics are specified; actual findings/counts not supplied.	cppcheck/clang-tidy report, LOC and complexity results.
v1.0 deployment	Deployment plan and manual outline supplied; completed installation evidence not supplied.	Deployment diagram, installation record, backup/restore test and release tag.
CR1 / v1.1	Impact analysis and test plan supplied; completion not verified.	Migration, implemented class extension, regression results and v1.1 tag.